Somos F5

Consumo de APIs y Protocolos de Comunicación Web

Fundamentos de HTTP, Estructura RESTful y Manejo de Datos.

ABRIMOS RUTAS INCLUSIVAS AL TALENTO DIGITAL

Hoja de ruta detallada

DÍA 1	Fundamentos y conceptos	• APIs • RESTful • Stateless
	El lenguaje de la web	• Métodos `HTTP`
	Los datos	• El formato JSON y su traducción a diccionarios
DÍA 2	Los datos	• Librería `requests`
	Gestión de respuestas	• Códigos de estado • Manejo de errores (`try/except`)
	Seguridad y autenticación	• Parámetros de consulta • API Keys

¿Qué es una API?

¿Qué es una API?

- **Application Programming Interface** (Interfaz de Programación de Aplicaciones).
- Un conjunto de definiciones y protocolos que permiten que dos aplicaciones de software se comuniquen entre sí.
- Actúa como una capa de abstracción: no necesitas saber cómo funciona el sistema interno, solo cómo pedirle los datos.

Ejercicio de reflexión: ¿Con cuántas APIs interactúas en un día?

5 minutos.
Crea un listado completo.
Cada persona compartirá 1 ejemplo.
¡Escucha a tus compis porque no vale repetir!

El origen de las RESTful APIs

El origen de las RESTful APIs

- ¿Bajo qué arquitectura específica funcionan las RESTful APIs?
- ¿Qué significa que el cliente y el servidor sean "cajas negras"?
- ¿Qué protocolo de comunicación es el más utilizado?
- ¿Qué significa que una API sea "Stateless"?
- ¿Por qué es importante para la escalabilidad?

El origen de las RESTful APIs

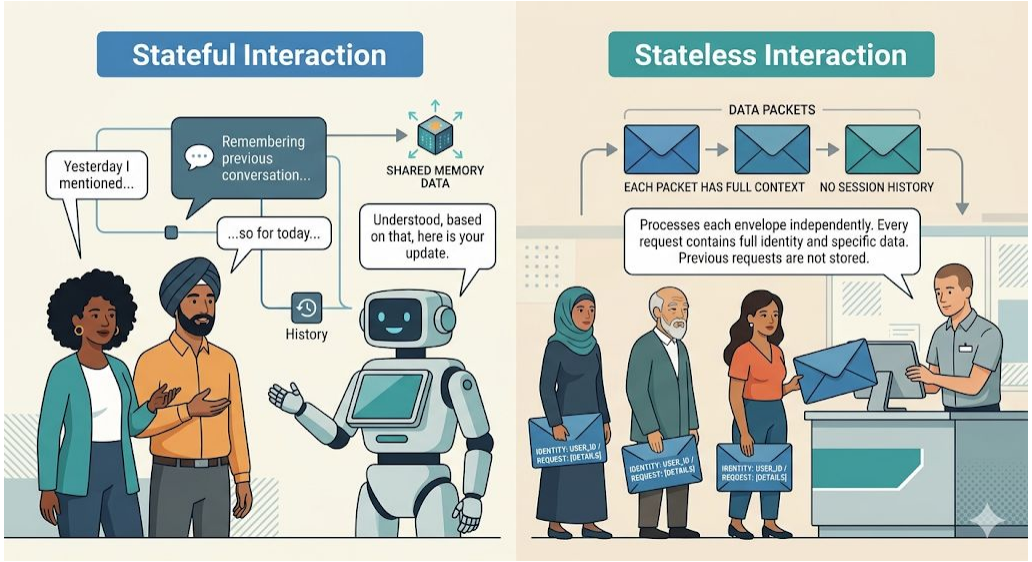
- **REST: Representational State Transfer** (Transferencia de Estado Representacional). Introducido por Roy Fielding en el año 2000.
- Modelo basado en la simplicidad y el uso de los estándares web.
- Arquitectura cliente-servidor.
- **Protocolo**: Las APIs en general pueden usar diversos protocolos (SOAP, gRPC, etc.). Las API REST utilizan casi exclusivamente **HTTP/HTTPS**.
- **Formato de Datos**: REST suele ser más flexible y ligero, utilizando mayoritariamente JSON o XML para el intercambio de datos.

Todas las RESTful APIs son APIs, pero no todas las APIs son RESTful.

El concepto “stateless” (sin estado)

- Cada solicitud de un cliente al servidor es independiente y debe contener toda la información necesaria para comprender y procesar la petición.

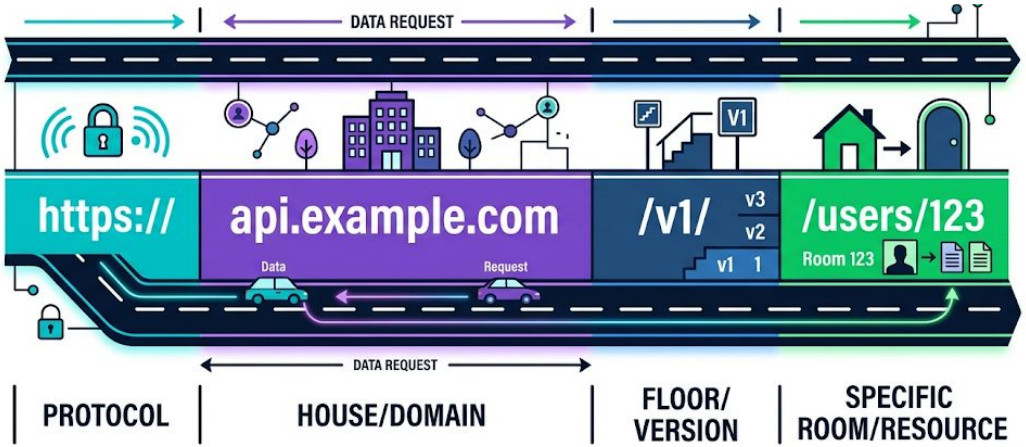
- El servidor no guarda memoria de las peticiones anteriores.



¿Qué ventajas crees que tiene una Stateless Interaction?

Recursos y URIs

- **Recurso**: Cualquier dato u objeto que la API puede proporcionar o manipular.
- **URI** (Uniform Resource Identifier): identificador único de un recurso.



¿De qué partes se compone esta URI?

Actividad de reflexión: URIs

¿Qué recurso se está solicitando?

- https://api.musica.com/v2/artistas/daft-punk
- https://api.clima.com/v1/ciudades/madrid/pronostico
- https://api.tienda.com/v3/pedidos/998/articulos

Métodos HTTP principales

- **GET**: Recuperar información. No modifica el servidor. Es seguro e "idempotente" (puedes hacerlo mil veces y el resultado es el mismo).
- **POST**: Crear un nuevo recurso. Envía datos en el cuerpo de la petición.
- **PUT**: Actualización completa. Reemplaza el recurso actual con una versión nueva.
- **PATCH**: Actualización parcial. Solo cambia los campos específicos indicados.
- **DELETE**: Elimina el recurso especificado por la URI.

Métodos de diagnóstico

- **HEAD**: Similar a GET, pero solo devuelve los encabezados (metadatos), no el cuerpo. Útil para verificar si un archivo ha cambiado sin descargarlo.

- **OPTIONS**: Pregunta al servidor qué métodos HTTP están permitidos para ese recurso específico. Es fundamental para la seguridad en navegadores (CORS).

Actividad de reflexión: selección de verbos

¿Qué método HTTP usarías para las siguientes acciones?

- **Cambiar la contraseña de tu cuenta.**
- **Borrar una foto de tu perfil.**
- **Buscar vuelos disponibles para mañana.**
- **Crear una nueva publicación en una red social.**

Representaciones de datos: el JSON

JSON: JavaScript Object Notation.

Formato ligero de intercambio de datos, legible tanto para personas como para máquinas. Basado en pares Clave-Valor.

```
{
"id": 123,
"nombre": "Juan Pérez",
"email": "juan@ejemplo.com"
}
```

¿A qué tipo de estructura de datos de Python te recuerda?

Laboratorio

Hoja de ruta detallada

DÍA 1	**Fundamentos y conceptos**	• APIs • RESTful • Stateless
	El lenguaje de la web	• Métodos `HTTP`
	Los datos I	• El formato JSON y su traducción a diccionarios
DÍA 2	**Los datos II**	• Librería `requests`
	Gestión de respuestas	• Códigos de estado • Manejo de errores (`try/except`)
	Seguridad y autenticación	• Parámetros de consulta • API Keys

La librería requests en Python

La herramienta estándar para comunicación HTTP que nos permite extraer datos fácilmente.

Instalación: `pip install requests` (disponible en Google Colab).

```
import requests
respuesta = requests.get("https://api.ejemplo.com/datos")
```

Ventaja: permite gestionar de forma automática los encabezados, la codificación y la persistencia de la conexión. Abstrae la complejidad del protocolo y gestiona automáticamente la conexión y codificación

De JSON a diccionario de Python

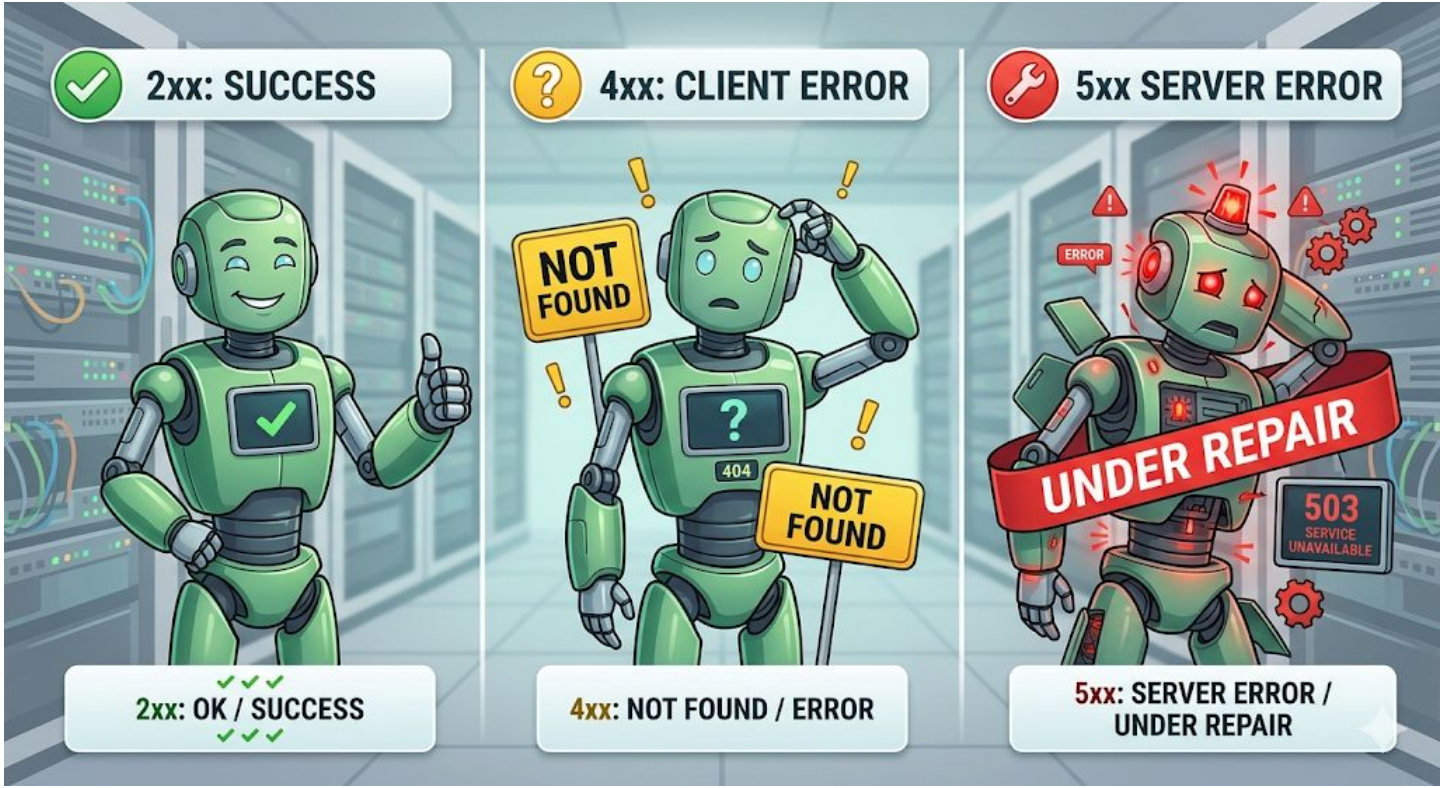
El método `.json()` de la librería `requests`.

Convierte automáticamente la respuesta de texto plano en una estructura de datos nativa de Python.

```
datos = respuesta.json()
print(type(datos))  # <class 'dict'> o <class 'list'>
```

Una vez convertido, podemos manipular la información usando las claves y valores que ya conocemos de los diccionarios.

Códigos de Estado



Códigos de Estado

2xx (Todo ha ido bien)

- 200 OK: Éxito general.
- 201 Created: El recurso se ha creado correctamente (respuesta típica a un POST).
- 204 No Content: Éxito, pero no hay nada que mostrar (típico tras un DELETE).

3xx (Redirecciones)

- 301 Moved Permanently: El recurso se ha movido a otra URI.
- 304 Not Modified: Usa tu copia local (caché), los datos no han cambiado.

4xx (Error del Cliente)

- 400 Bad Request: La solicitud es inválida
- 401 Unauthorized: Se requiere autenticación
- 403 Forbidden: El servidor entendió la solicitud pero se niega a autorizarla
- 404 Not Found: El recurso solicitado no se encontró

5xx (Error del Servidor)

- 500 Internal Server Error: Error genérico del servidor
- 503 Service Unavailable: El servidor no está disponible temporalmente

Actividad de reflexión: detectives de códigos

¿Qué código de estado recibirías si...?

- **Intentas ver una página que ya no existe.**
- **Envías un formulario de registro sin poner el email.**
- **Tratas de borrar un mensaje pero no has iniciado sesión.**
- **El servidor se ha quedado sin memoria.**

Control de errores

¿Qué pasa cuando la API falla? Evitando que el programa se detenga.

Estructura `try/except` para evitar que el programa “explote”

```
try:
    res = requests.get(url)
    res.raise_for_status() # Lanza error si no es 200 OK
except Exception as e:
    print(f"Error en la petición: {e}")
```

El manejo de excepciones permite que el programa continúe su ejecución o informe del error de forma controlada.

Parámetros de consulta (Query Params)

Somos F5

Permiten filtrar, ordenar o paginar los resultados. Se añaden al final de la URI tras el signo **?.**
Estructura: ?clave=valor&otra_clave=otro_valor Ejemplos:

- **Filtrar**: /usuarios?rol=admin
- **Paginación**: /usuarios?page=2&limit=20
- **Búsqueda**: /usuarios?q=juan

Implementación con `requests`: En lugar de construir el texto a mano, pasamos un diccionario al argumento params:

```
filtros = {
    'rol': 'admin',
    'page': 2
}
# Requests construye la URL completa por ti
respuesta = requests.get(url, params=filtros)
```

Seguridad y autenticación

Las APIs suelen estar protegidas. Métodos comunes:

- **API Keys**:
  - Una cadena de texto única que identifica al cliente.
  - Se suele pasar en la URL o en los encabezados (Headers).
  - Ejemplo: GET /datos?api_key=xyz123
- **Tokens**:
  - Credenciales temporales que otorgan permisos específicos por un tiempo limitado.

Seguridad y autenticación

Característica	API Key	OAuth Token (JWT)
¿A quién identifica?	Al proyecto/aplicación.	Al usuario específico.
Vida útil	Larga (meses o años).	Corta (minutos u horas).
Seguridad	Baja (si se filtra, es un problema).	Alta (es revocable y temporal).
Uso común	Proyectos pequeños / Datos públicos.	Aplicaciones Full-stack / Datos privados.

Laboratorio

Somos F5

www.somosf5.org

Síguenos en